

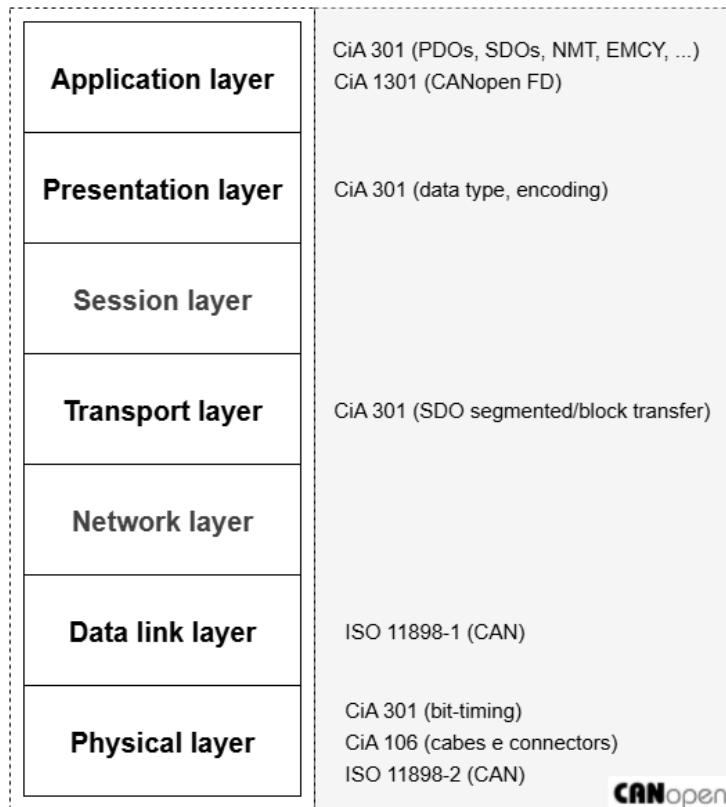
Introdução ao CANopen

Autor(a): Mercedes Diniz

Como alternativa viável e mais acessível, destaca-se o protocolo CANopen, uma solução de código aberto baseada na tecnologia *Controller Area Network* (CAN). Desenvolvido e mantido pela organização internacional sem fins lucrativos *CAN in Automation* (CiA), o mesmo oferece robustez, confiabilidade, imunidade a ruído, acesso não destrutivo ao barramento, configuração flexível, priorização de mensagens, mecanismos de detecção e tratamento de erros [1].

O protocolo CANopen pode ser representado com base no modelo OSI, conforme ilustrado na Figura 1. Nesse contexto, a especificação CiA 301 [2] define os elementos fundamentais da camada de aplicação, incluindo os modos de comunicação suportados (como master/slave, client/server e producer/consumer), os *Communication Objects* (CO), os estados dos dispositivos, o *Object Dictionary* (OD) e os *Device Profiles*. Já a CiA 306-1 [3], por sua vez, descreve o formato dos arquivos EDS (*Electronic Data Sheet*) e DCF (*Device Configuration File*), que são essenciais para a configuração e interoperabilidade dos dispositivos. Além disso, a CiA 106 [4] trata das recomendações para a atribuição de pinos em conectores físicos para os dispositivos da rede.

Figura 1 - CANopen conforme o modelo de referência ISO-OSI.



Fonte: Adaptado de [1] e [2].

O CANopen Clássico (CC) é baseado no protocolo CAN Clássico, conforme a norma ISO 11898-1, utilizando uma camada física tipicamente conforme a ISO 11898-2, embora outras opções também sejam suportadas. Ele opera com taxa de transmissão ajustável entre 10 kbit/s e 1000 kbit/s e utiliza predominantemente endereçamento 11 bits, mas também oferece suporte ao formato de 29 bits. Já o CANopen FD é uma versão mais recente e avançada, baseada no CAN FD (*Flexible Data Rate*), oferecendo maior desempenho e capacidade de dados [2] [5].

1. Dicionário de Objetos

Em uma rede CANopen, cada dispositivo possui um Dicionário de Objetos (*Object Dictionary* – OD), que é uma estrutura padronizada contendo todos os parâmetros que descrevem o seu funcionamento. Nesse dicionário ficam armazenadas as configurações do dispositivo, os dados operacionais, as informações de diagnóstico e os mapeamentos de dados utilizados na comunicação.

As entradas do OD são organizadas por meio de um índice de 16 bits e um subíndice de 8 bits, o que garante a identificação única de cada parâmetro. Cada

objeto é descrito por atributos como: índice, nome, código do objeto, tipo de dado, direitos de acesso e categoria. O dicionário é dividido em seções padronizadas, como exemplo apresentado na Tabela 1, nas quais algumas entradas são obrigatórias, enquanto outras podem ser totalmente personalizadas pelo fabricante ou pela aplicação [2].

Tabela 1 - Seções padronizadas do OD.

Índice	Objeto
0x0000	não utilizado
0x0001 – 0x001F	Tipos de dados estáticos
0x0020 – 0x003F	Tipos de dados complexos
0x0040 – 0x005F	Tipos de dados complexos específicos do fabricante
0x0060 – 0x025F	Tipos de dados específicos do perfil do dispositivo
0x0260 – 0x0FFF	reservado
0x1000 – 0x1FFF	Área do perfil de comunicação
0x2000 – 0x5FFF	Área do perfil específico do fabricante
0x6000 – 0x9FFF	Área de perfil padronizado dos dispositivos lógicos
0xA000 – 0xAFFF	Área de variáveis de rede padronizadas
0xB000 – 0xBFFF	Área de variáveis de sistema padronizadas
0xC000 – 0xFFFF	reservado

Para simplificar a configuração e o gerenciamento de dispositivos em redes CANopen, a especificação CiA 306-1 define o formato de arquivo INI usado na *Electronic Data Sheet* (EDS). O EDS funciona como um modelo do OD de um dispositivo, descrevendo todos os objetos disponíveis, incluindo os parâmetros suportados e os mapeamentos padrões [3].

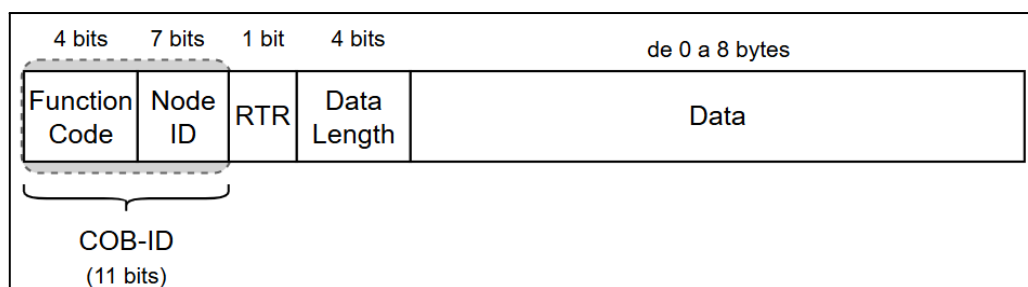
Além disso, existe o *Device Configuration File* (DCF), que é uma versão do EDS adaptada para uma aplicação específica. O DCF incorpora detalhes de integração, como a taxa de transmissão (*bit rate*) e o identificador do dispositivo, sendo o arquivo efetivamente editado durante a configuração da rede [3].

2. Objetos de Comunicação

Os frames CANopen são baseados na estrutura dos frames CAN, uma vez que o CANopen atua como um protocolo de camada superior que utiliza o barramento CAN como meio físico de comunicação. Nesse sentido, a “estrutura” de um frame CANopen corresponde à forma como os campos de um frame CAN subjacente são interpretados e utilizados. O tamanho do payload disponível, por sua vez, depende do tipo de barramento adotado, podendo ser limitado ao CAN Clássico ou CAN FD [1][2].

No caso do CAN Clássico, a estrutura básica de um frame CANopen está ilustrada na Figura 2, cujo o principal campo é o *Communication Object Identifier* (COB-ID), um identificador de 11 bits que se divide em duas partes: 4 bits para o código de função, que indica o tipo de objeto de comunicação (conforme listado na Tabela 2), e 7 bits para o Node-ID, que identifica o dispositivo transmissor da mensagem, com valores variando de 1 a 127. Cada dispositivo em uma rede deve possuir um endereço único e, embora o limite teórico seja de 127 nós, na prática a quantidade de dispositivos conectados depende das características do *transceiver* utilizado. Redes típicas operam com até cerca de 63 dispositivos, mas esse número pode ser ampliado mediante o uso de repetidores [1][2].

Figura 2 - Frame CANopen.



Fonte: Os autores.

Existem sete tipos de *Communication Objects* (COs) definidos no CANopen CC, conforme apresentado na Tabela 2. Nessa tabela, é possível observar a prioridade dos COs na disputa pelo barramento, o modo de comunicação utilizado na transmissão e o código correspondente de cada CO.

Tabela 2 - Tipos de *Communication Object* do CANopen.

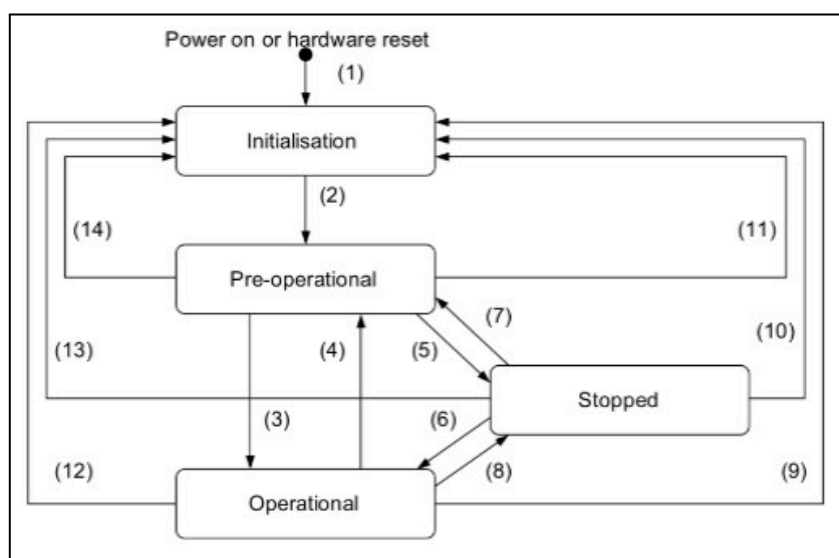
CO	Prioridade	Modo de comunicação	Código de Função		ID do nó (dec)	Intervalo de COD-IDs
NMT	1	Master/Slave	0000	0x0	<i>Broadcast/</i> 1-127	0x0 (broadcast)
SYNC	2	Producer/Consumer	0001	0x1	<i>Broadcast</i>	0x80
EMCY	3	Producer/Consumer	0001	0x1	1-127	0x81-0xFF
TIME	4	Producer/Consumer	0010	0x2	<i>Broadcast</i>	0x100
PDO	5	Producer/Consumer	*	*	1-127	0x181-0x57F
SDO	6	Client/Server	*	*	1-127	0x581*0x67F
HEARTBEAT	7	Producer/Consumer	1110	0xE	1-127	0x710-0x77F

* Mais detalhes na Tabela 4.

2.1 NMT (Network Management Objects)

O gerenciamento de rede CANopen é coordenado por um dispositivo controlador, que desempenha o papel de master NMT (Network Management Tools). Esse dispositivo é responsável por gerenciar o estado operacional dos nós slaves através dos comandos NMT, seguindo uma máquina de estados finitos, conforme ilustrado na Figura 3. O ciclo de operação de cada nó começa no estado de *Initialisation*, o qual ocorre logo após a inicialização ou um reset do dispositivo. Quando um nó sai desse estado, ele transmite uma mensagem Boot-up (um Heartbeat com payload 0x00), sinalizando que está pronto para transitar para o estado *Pre-operational* [2].

Figura 3 - Diagrama de estado NMT de um dispositivo CANopen.



Fonte: Adaptado de [4].

No estado *Pre-operatinal*, o nó fica disponível para configuração e diagnóstico via SDOs (*Service Data Object*). Quando o dispositivo entra no estado *Operational*, ele atinge o seu funcionamento normal, sendo esse o estado no qual as funcionalidades principais da aplicação ficam ativas, permitindo o processamento e transmissão de dados em tempo real através dos PDOs (*Process Data Objects*). Além disso, o nó continua respondendo aos SDOs para manutenção e ajustes de configuração [2].

Por outro lado, o estado *Stopped* é um modo de segurança ou de baixa atividade, no qual o nó realiza apenas o processamento de mensagens de gerenciamento de rede (NMT) [2].

Os comandos NMT são transmitidos com um código de função 0x0 e incluem um payload de dois bytes. O primeiro byte especifica o código do estado para o qual o nó slave deve ser alterado, conforme os valores listados na Tabela 3. O segundo byte contém o ID do nó ao qual o comando NMT é direcionado, e caso esse valor seja 0x0 o comando é entendido como um *broadcast*, afetando todos os nós na rede simultaneamente [2].

Tabela 3 - Códigos dos comandos que o master NMT pode enviar para os nós slaves.

Estado solicitado	Código
Operational	0x01
Stopped	0x02
Pre-operation	0x80
Reset Application	0x81
Reset Communication	0x82

2.2 SYNC (*Synchronization Object*)

O dispositivo controlador da rede CANopen não apenas gerencia a comunicação, mas também é responsável por transmitir periodicamente mensagens SYNC, sendo esse intervalo de tempo configurável. Essas mensagens fornecem um mecanismo

básico de sincronização entre os dispositivos da rede, permitindo que os nós consumidores de SYNC alinhem o momento exato para a transmissão de seus dados.

Após as mensagens NMT, os SYNC são as mensagens de maior prioridade, com um COB-ID de 0x80. Por padrão, as mesmas não transportam dados, no entanto com a introdução da versão 4.1 da norma CiA 301, passou a ser possível incluir opcionalmente um contador SYNC de 1 byte, agregando uma funcionalidade adicional à mensagem [2].

2.3 EMCY (Emergency Object)

Quando um dispositivo detecta um erro fatal, ele gera uma mensagem EMCY (*Emergency Object*) para informar sobre a falha. Essa mensagem é transmitida uma única vez por evento de erro, sendo gerada pelo nó que detectou o problema interno. A mensagem possui um código de função 0x80, e o COB-ID é formado pela combinação desse código com o node ID do dispositivo.

O payload da mensagem EMCY contém informações detalhadas sobre o erro e isso inclui um código de erro de 2 bytes, que fornece uma identificação do tipo de falha ocorrida. A tabela de códigos de erro é definida pela especificação CiA 301, e códigos adicionais podem ser especificados em perfis específicos de dispositivos. Além disso, a mensagem inclui um registro de erro de 1 byte e, em alguns casos, até 5 bytes com informações adicionais de erro específicas do fabricante [1][2].

Essas informações de erro são fundamentais para diagnóstico e manutenção, e os códigos de erro de emergência estão listados na Tabela 4.

Tabela 4 - Códigos de erro de emergência (EMCY) definidos na CiA 301.

Código de erro	Descrição	Código de erro	Descrição
0x0000	Reinício de erro / nenhum erro	0x7000	Módulos adicionais
0x1000	Erro genérico	0x8000	Monitoramento
0x2000	Corrente	0x8100	– Comunicação
0x2100	– Lado de entrada do dispositivo	0x8110	– CAN overrun (objetos perdidos)
0x2200	– Dentro do dispositivo	0x8120	– CAN em modo passivo de erro
0x2300	– Lado de saída do dispositivo	0x8130	– Lifeguard/heartbeat

0x3000	Tensão	0x8140	– Recuperado de bus-off
0x3100	– Tensão da rede elétrica	0x8150	– Colisão de ID CAN
0x3200	– Dentro do dispositivo	0x8200	– Protocolo
0x3300	– Tensão de saída	0x8210	– PDO não processado devido ao comprimento
0x4000	Temperatura	0x8220	– Comprimento do PDO excedido
0x4100	– Temperatura ambiente	0x8230	– DAM MPDO não processado
0x4200	– Temperatura do dispositivo	0x8240	– Comprimento de dados SYNC inesperado
0x5000	Hardware do dispositivo	0x8250	– Timeout de RPDO
0x6000	Software do dispositivo	0x9000	Erro externo
0x6100	– Software interno	0xF000	Funções adicionais
0x6200	– Software do usuário	0xFF00	Específico do dispositivo
0x6300	– Conjunto de dados	-	

2.4 TIME (*Timestamp Object*)

O TIME (*Timestamp Object*) é outro *Communication Object* crucial relacionado à temporização, fornecido pelo dispositivo controlador da rede CANopen. Sua principal função é disponibilizar um relógio sincronizado para toda a rede, distribuindo um timestamp global para garantir que todos os dispositivos compartilhem uma referência temporal comum.

O produtor do TIME transmite essa mensagem periodicamente, com o intervalo entre as transmissões sendo determinado pelo próprio produtor, de acordo com as necessidades da aplicação. A mensagem TIME é transmitida com um COB-ID de 0x100, configurado para broadcast, e possui um payload de 6 bytes. Desses, os primeiros 4 bytes indicam o tempo em milissegundos após a meia-noite, enquanto os 2 bytes seguintes registram o número de dias desde 1º de janeiro de 1984 [1][2].

A principal diferença entre as mensagens TIME e SYNC está em seus objetivos: enquanto o SYNC funciona como um "pulso" para coordenar a execução de tarefas em tempo específico, o TIME serve como uma referência contínua e global de tempo.

2.5 HEARTBEAT (*Heartbeat Object*)

As mensagens HEARTBEAT (*Heartbeat Object*) são as de menor prioridade na rede CANopen e seguem uma dinâmica diferente das demais mensagens descritas até aqui. Nessa comunicação, a relação de produtor e consumidor é invertida: enquanto o dispositivo controlador da rede atua como consumidor, todos os outros dispositivos na rede são os produtores das mensagens, enviando-as periodicamente para informar seu estado.

O principal objetivo do *Heartbeat* é monitorar e garantir que todos os dispositivos na rede mantenham o estado NMT para o qual foram configurados. Essas mensagens são transmitidas com um COB-ID de 0x700 somado ao *node ID* do dispositivo que as envia, e o *payload* consiste em um único byte de dados, que contém o estado NMT atual do nó.

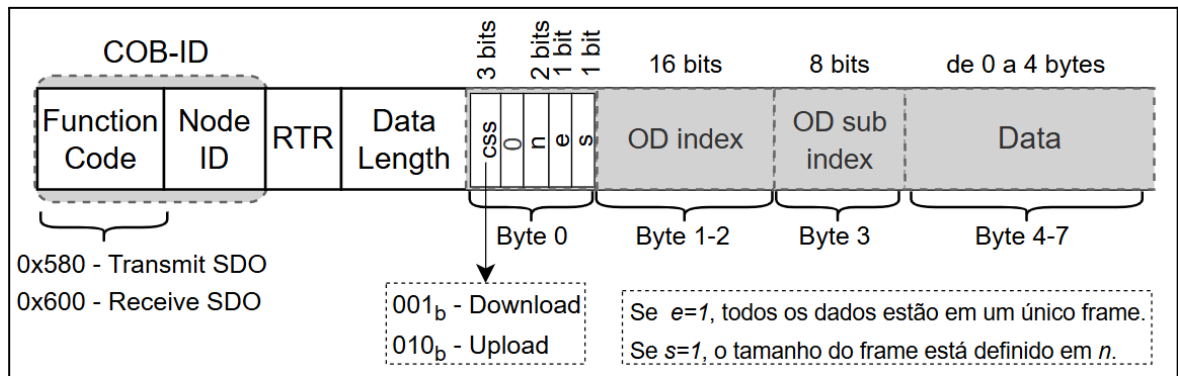
Este serviço é ativado ao definir o parâmetro *heartbeat-producer-time* no dicionário de objetos para um valor diferente de zero. Esse valor define o intervalo de tempo entre as transmissões das mensagens. Como regra geral, o *timeout* do consumidor deve ser o dobro do período de produção das mensagens, para garantir uma monitoração adequada da rede.

2.6 SDO (*Service Data Object*)

O *Service Data Object* (SDO) é o objeto de comunicação que permite o acesso remoto ao *Object Dictionary* de um dispositivo. Por meio dele, o dispositivo gerenciador da rede pode ler (*SDO upload*) ou escrever (*SDO download*) parâmetros nos ODs de outros nós, o que proporciona flexibilidade na configuração e operação de toda a rede.

A comunicação ocorre em um modelo de *client-server*, onde o cliente SDO, normalmente associado ao nó controlador, inicia a transmissão, enquanto o servidor SDO, que corresponde ao dispositivo cujo OD está sendo acessado, responde de acordo com a requisição. Para distinguir as mensagens, são utilizados código de funções específicas, com 0x580 para o envio das solicitações do cliente para o servidor (*Transmit SDO*) e, 0x600 quando o servidor responde às solicitações do cliente (*Receive SDO*) .

Figura 4 - Frame SDO.



Fonte: Os autores

A estrutura do frame SDO pode ser observada na Figura 4, onde o primeiro byte contém informações de controle, incluindo o *Client Command Specifier* (CCS), que define o tipo de operação, além de bits adicionais que indicam o número de bytes não utilizados (n), se a transferência é rápida com todos os dados em um único frame (e) e se o tamanho dos dados está explicitamente especificado (s). Em seguida, dois bytes identificam o índice do objeto no OD e um terceiro byte indica o subíndice, permitindo a localização exata do parâmetro desejado dentro do OD. Finalmente, os quatro últimos bytes podem transportar o dado propriamente dito, que em operações de *download* pode permanecer vazio, mas em operações de *upload* traz o valor solicitado ao servidor.

2.7 PDO (Process Data Object)

O PDO (*Process Data Object*) é um dos principais mecanismos de comunicação do protocolo CANopen, desempenhando um papel essencial na transferência eficiente de dados operacionais em tempo real entre os nós da rede. Sua principal função é garantir que informações críticas, como estados de sensores ou comandos para atuadores, circulem rapidamente e com baixa latência.

As mensagens PDO podem ser transmitidas de forma síncrona, em resposta a uma mensagem de sincronização (SYNC), ou de maneira assíncrona, disparadas por eventos específicos, como alterações de estado ou intervalos de tempo programados. A comunicação segue o modelo *producer-consumer*, no qual um nó produtor gera e transmite os dados para a rede por meio de um *Transmit PDO* (TPDO), enquanto um

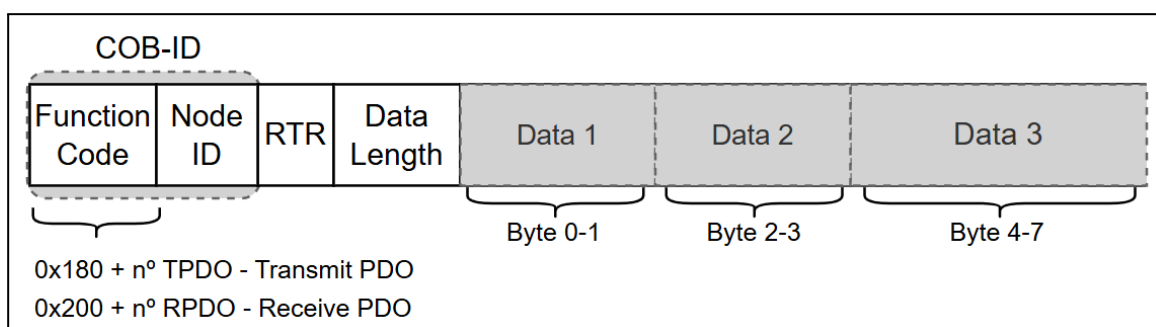
ou mais nós consumidores recebem e utilizam esses dados por meio de um *Receive PDO (RPDO)*.

Outro aspecto relevante é o mapeamento de PDOs (*PDO mapping*), que determina quais variáveis do OD são agrupadas e transmitidas dentro de um mesmo frame, ilustrado na Figura 5. Essa configuração é flexível e permite ao projetista da rede otimizar a comunicação de acordo com as necessidades específicas da aplicação.

No CANopen CC, o *payload* de um frame PDO pode conter até 8 bytes de dados, respeitando a limitação do CAN CC. Já no CANopen FD, é possível aproveitar as extensões do CAN FD, o que amplia a capacidade do payload para até 64 bytes, permitindo a transmissão de conjuntos de dados mais extensos em uma único frame.

Cada dispositivo CANopen possui uma configuração genérica predefinida para os COB-IDs associados aos seus PDOs. Para os TPDOs, o identificador é formado pela soma dos valores 0x180, *node ID* e o número do TPDO, enquanto para os RPDOs o soma é dos valores 0x200, *node ID* e o número do RPDO. Tanto o número do TPDO quanto o RPDO podem assumir valores de 1 a 4, o que define os quatro tipos possíveis de PDOs, estando esses listados na Tabela 4. Na prática, é comum personalizar esses identificadores para garantir que os consumidores corretos recebam as mensagens.

Figura 5 - Frame PDO.



Fonte: Os autores

Tabela 4 - Formato do código de função do *Process Data Object*.

CO	Código de Função	
Transmit PDO 1	0011	0x180

Receive PDO 1	0100	0x200
Transmit PDO 2	0101	0x280
Receive PDO 2	0110	0x300
Transmit PDO 3	0111	0x380
Receive PDO 3	1000	0x400
Transmit PDO 4	1001	0x480
Receive PDO 4	1010	0x500

3. Perfis de Dispositivos

Os *device profiles* definidos pela CiA são uma padronização do comportamento e da interface de determinados tipos de dispositivos dentro de uma rede CANopen garantindo a interoperabilidade. A ideia central é que, em vez de cada fabricante definir de forma proprietária como acessar variáveis, parâmetros e funcionalidades do seu equipamento, a CiA define quais entradas do OD são padronizadas e como devem ser acessadas em todos os dispositivos de uma mesma classe devem implementar.

Na Tabela 5 estão listados alguns desses *device profiles* que podem estar relacionados com os possíveis dispositivos que iram compor a rede CANopen da embarcação.

Tabela 5 - Alguns *device profiles* definidos pela CiA.

Norma	Descrição
CiA 401 series	CANopen profile for I/O device
CiA 418	CANopen device profile for battery modules
CiA 419	CANopen device profile for battery chargers
CiA 453	CANopen device profile for power supplies
CiA 458	CANopen device profile for energy measures
CiA 456	CANopen interface profile for configurable network components
CiA 457	CANopen interface profile wireless transmission
CiA 447 series	CANopen application profile for special-purpose car add-on devices
CiA 454 series	CANopen application profile for energy management systems

4. Topologia Básica

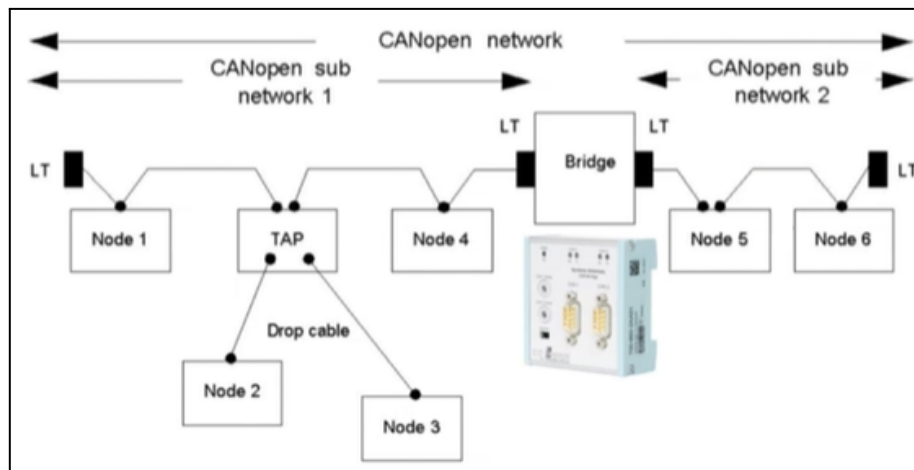
Na Tabela 6, estão listados os elementos comuns que compõem uma rede CANopen, qual a topologia básica é formada por um cabo tronco principal, chamado *trunk cable*, que funciona como o *backbone* da comunicação. Esse cabo deve ser terminado em ambas as extremidades físicas com resistores de 120 Ω , garantindo compatibilidade com a impedância característica da linha e evitando reflexões que poderiam comprometer a transmissão dos dados [7].

Tabela 6 - Elementos da arquitetura de uma rede CANopen.

Elemento	Descrição
Bus Cable	Forma o <i>backbone</i> da rede e em ambas as extremidades devem ter resistores de terminação.
Stub/Drop Cable	Ramificação curta ligada ao barramento principal.
T-connector/TAP	Ligar dispositivos ao cabo <i>trunk cable</i> .
Trunk Cable	Linha principal do barramento.
Repetidor	Regenera o sinal CAN e o retransmite (atua na camada física).
Bridge	Conecta segmentos da rede (atua na camada de enlace e de aplicação).

Os dispositivos são conectados ao cabo principal por meio de *T-connectors* ou através de ramificações curtas chamadas *stub/drop cables*. Em conjunto com os *TAPs*, esses ramais possibilitam a criação de uma estrela parcial, exemplificada na Figura 5, o que facilita a conexão de múltiplos nós sem interromper o cabo principal. No entanto, é fundamental que essas derivações sejam mantidas o mais curtas possível, já que cabos longos tendem a gerar reflexões indesejadas e perda de integridade no sinal [7].

Figura 5 - Exemplo da topologia de uma rede CANopen.



Além disso, a rede pode contar com repetidores, que regeneram o sinal CAN, embora introduzem atrasos de propagação e reduzam no comprimento máximo do barramento, e com bridges, que conectam diferentes segmentos da rede, podendo até operar com taxas de transmissão distintas. Para a conexão física, a CiA 106 [4] recomenda o uso dos conectores padronizados D-Sub de 9 pinos (DB9), ou M12 de 5 pinos, além da escolha adequada de cabos conforme a taxa de transmissão, bitola mínima e requisitos de impedância.

REFERÊNCIAS

- 1 **CANopen Explained - A Simple Intro [2025]**. Disponível em: <https://www.csselectronics.com/pages/canopen-tutorial-simple-intro>. Acesso em: 25 jun. 2025.
- 2 **CiA 301: CANopen application layer and communication profile**. Disponível em: <https://www.can-cia.org/cia-groups/technical-documents>. Acesso em: 5 ago. 2025.
- 3 **CiA 306-1: CANopen electronic data sheet (EDS)**. Disponível em: <https://www.can-cia.org/cia-groups/technical-documents>. Acesso em: 5 ago. 2025.
- 4 **CiA 106: Connector pin-assignment recommendations**. Disponível em: <https://www.can-cia.org/cia-groups/technical-documents>. Acesso em: 5 ago. 2025.
- 5 **CANopen CC – The standardized embedded network**. Disponível em: <https://www.can-cia.org/can-knowledge/canopen>. Acesso em: 25 jun. 2025.
- 6 **CANopen EDS File Explained - A Simple Intro [2025]**. Disponível em: <https://www.csselectronics.com/pages/canopen-eds-file-electronic-data-sheet>. Acesso em: 25 jun. 2025.

7 CiA 303-1: Device and network design - Part 1: CANopen physical layer.

Disponível em: <<https://www.can-cia.org/cia-groups/technical-documents>>. Acesso em: 5 ago. 2025.